The Calibration Audit

Drivers + Shell in the Field — Live Meeting 3



The Story

Iomico's environment monitors log temperature in a pharmacy stockroom. The prototypes used **AHT20** sensors, with Zephyr's upstream `aosong,aht20` driver.

For production, purchasing switched to cheaper **AHT10** modules. Same pinout, same address, and the upstream driver happened to read them. 200 units shipped.

Then the audit. Next to a calibrated reference thermometer, the units read **0.5 to 2 °C high**. Every unit must be within ±0.5 °C, fixed **on site**. Nobody is reflashing 200 devices.

The next audit is Monday.



What This Demo Ties Together



- **Drivers** (L06): binding, Kconfig, CMake, Sensor API, instantiation
- **Shell** (L07): built-in commands, static subcommands, custom handlers
- **New tonight**: how to **extend** the Sensor API without breaking it



Objectives



- **Check** a sensor layer by layer with the built-in shell
- **Write** a Sensor API driver when upstream doesn't do what you need
- **Extend** the Sensor API: attributes, private channels, device-specific functions
- **Calibrate** a sensor on site from the shell, with no reflash
- **Build** custom shell commands for the extensions

The Rig

Hardware

- FRDM-MCXN236
- AHT10 module on the Arduino header I2C
  - SDA → P1_16, SCL → P1_17
  - VCC → 3V3, GND → GND
- `flexcomm5_lpi2c5`, 100 kHz

Repo: `zephyr-fundamentals-lm3/`

```
app/                    # the product
app/drivers/sensor/     # the driver
```

Git tags, one per act

```
step-1-diagnose
step-2-driver
step-3-extend
step-4-shell
```

`step-1-diagnose`



Act 1 — Can the Shell Fix It?

What We Ship Today

```
/* app/boards/frdm_mcxn236.overlay */
&flexcomm5_lpi2c5 {
          env_sensor: aht@38 {
                    compatible = "aosong,aht20";
                    reg = <0x38>;
                    status = "okay";
          };
};
```

```
# prj.conf
CONFIG_SHELL=y
CONFIG_I2C=y
CONFIG_I2C_SHELL=y      # built-in: i2c scan / read / write
CONFIG_SENSOR=y
CONFIG_SENSOR_SHELL=y   # built-in: sensor get / attr_set
```



Check One Layer at a Time



Layer	Question	Tool
Bus	Is anything answering at 0x38?	`i2c scan`
Device	Did the driver's init succeed?	`device list`
Driver	Does the API return data?	`sensor get`

Start at the bottom. Never guess past a layer you haven't checked.

Live: Walk the Layers

```
uart:~$ i2c scan lpi2c@b5800
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
30: -- -- -- -- -- -- -- -- 38 -- -- -- -- -- -- --
1 devices found on lpi2c@b5800

uart:~$ device list
- lpi2c@b5800 (READY)
  DT node labels: flexcomm5_lpi2c5 p3t1755dp_ard_i2c_interface
- aht@38 (READY)
  DT node labels: env_sensor

uart:~$ sensor get aht@38
channel type=13(ambient_temp) ... (21.367644)
channel type=16(humidity) ... (62.290190)
```


Everything works. The readings are just wrong for this unit.



Live: Try the Obvious Fix



The Sensor API has a standard attribute for this: `SENSOR_ATTR_OFFSET`.

```
uart:~$ sensor attr_set aht@38 ambient_temp offset -1.5
Failed to set channel(ambient_temp) attribute(offset): -88
```

`-88` is `-ENOSYS`: **the driver doesn't implement `attr_set` at all.**

What Upstream Gives Us

What the product needs	Upstream `aosong,aht20`
Temperature and humidity	✅
Official AHT10 support	❌ DHT20, AHT20 and AM2301B only
Per-unit offset, set in the field	❌ no `attr_set`
Board-level offset from devicetree	❌
Dew point (condensation risk)	❌
Service commands: status, reset	❌

A good driver, for a **different product**.

Three Options

Option	Ships Monday?	Cost
Patch Zephyr's `dht20.c` in our tree	✅	We maintain a fork of Zephyr
Upstream the feature	❌ review takes weeks	None, long-term the best
Own driver in our app	✅	We maintain one driver, for **the part we actually ship**

Tonight: our own driver. Later: upstream what's reusable.

`step-2-driver`



Act 2 — The Driver

The Driver Lives in the App

```
app/
├── CMakeLists.txt
├── Kconfig
├── prj.conf
├── boards/frdm_mcxn236.overlay
├── dts/bindings/sensor/
│   └── aosong,aht10.yaml
├── drivers/sensor/aht10/
│   ├── CMakeLists.txt
│   ├── Kconfig          # CONFIG_AHT10
│   └── aht10.c
└── src/main.c
```

```
# app/CMakeLists.txt
add_subdirectory(drivers/sensor/aht10)

# drivers/sensor/aht10/CMakeLists.txt
target_sources_ifdef(CONFIG_AHT10 app
                     PRIVATE aht10.c)
```

```
# app/Kconfig
rsource "drivers/sensor/aht10/Kconfig"

source "Kconfig.zephyr"
```

`app/dts/bindings/` is found automatically: the app is already a DTS root.

App or Module?

	Driver in the app (tonight)	Driver in a module (L06)
Wiring	`add_subdirectory` + `rsource`	`zephyr/module.yml` + west manifest
Used by	This product only	Several apps / products
Versioned with	The app	Its own repo and releases
Good for	Bring-up, one-off parts, emergencies like tonight	Shared, reusable drivers

Start in the app. **Move it to a module when a second product needs it.**

Binding + Kconfig

```
# aosong,aht10.yaml
compatible: "aosong,aht10"

include:
  - sensor-device.yaml
  - i2c-device.yaml
```

Vendor prefix = the **manufacturer**, not us. `aosong` is already registered upstream.

```
config AHT10
          bool "Aosong AHT10 ..."
          default y
          depends on
DT_HAS_AOSONG_AHT10_ENABLED
          depends on SENSOR
          select I2C
```

Node present → driver compiled. Nothing to add to `prj.conf`.



The Overlay Changes, the App Doesn't




```
&flexcomm5_lpi2c5 {
        env_sensor: aht10@38 {
                compatible = "aosong,aht10";    /* was "aosong,aht20" */
                reg = <0x38>;
                status = "okay";
        };
};
```

`main.c` still asks for `DT_NODELABEL(env_sensor)`, with **zero lines changed**.



`sample_fetch` : Talk to the Chip




```
k_mutex_lock(&data->lock, K_FOREVER);

ret = aht10_write_cmd(dev, measure_cmd, sizeof(measure_cmd));  /* 0xAC 0x33 0x00 */
k_msleep(AHT10_MEASUREMENT_MS);                                /* 80 ms */
ret = i2c_read_dt(&cfg->i2c, frame, sizeof(frame));            /* 6 bytes + CRC */

if (frame[0] & AHT10_STATUS_BUSY) {
          ret = -EBUSY;
          goto out;
}
/* CONFIG_AHT10_CRC: crc8() over the 6 data bytes must match frame[6] */

/* [status][hum 19:12][hum 11:4][hum 3:0 | temp 19:16][temp 15:8][temp 7:0][crc] */
data->raw_humidity    = sys_get_be24(&frame[1]) >> 4;
data->raw_temperature = sys_get_be24(&frame[3]) & 0xFFFFF;
```



`channel_get` : Convert, No Floats




```c
/* RH [%] = raw / 2^20 * 100 */
static int64_t aht10_humidity_micro(const struct aht10_data *data)
{
        return ((int64_t)data->raw_humidity * 100000000) >> 20;
}

/* T [degC] = raw / 2^20 * 200 - 50 */
static int64_t aht10_temperature_micro(const struct aht10_data *data)
{
        return (((int64_t)data->raw_temperature * 200000000) >> 20) - 50000000;
}
```

```c
if (chan == SENSOR_CHAN_AMBIENT_TEMP) {
        return sensor_value_from_micro(val, aht10_temperature_micro(data));
}
```



Init + Instantiation




```
static int aht10_init(const struct device *dev)
{
          ...
          k_msleep(AHT10_POWER_ON_MS);                /* 100 ms */
          aht10_write_cmd(dev, reset_cmd, 1);         /* soft reset, 0xBA */
          k_msleep(AHT10_SOFT_RESET_MS);
          return aht10_chip_init(dev);                /* CAL bit set? done. Else 0xE1 */
}

#define AHT10_DEFINE(inst)                                              \
          static struct aht10_data aht10_data_##inst;                     \
          static const struct aht10_config aht10_config_##inst = {        \
                    .i2c = I2C_DT_SPEC_INST_GET(inst),                      \
          };                                                              \
          DEVICE_DT_INST_DEFINE(inst, aht10_init, NULL, ...,              \
                                POST_KERNEL, CONFIG_SENSOR_INIT_PRIORITY, &aht10_api)

DT_INST_FOREACH_STATUS_OKAY(AHT10_DEFINE)
```



Live: The First Payoff




```
git checkout step-2-driver
west build -p -b frdm_mcxn236 app && west flash
```

```
uart:~$ sensor get aht10@38
channel type=13(ambient_temp) ... (21.002387)
channel type=16(humidity) ... (63.979147)
```

We wrote **zero** shell code. `sensor get` works because we implemented the **standard API**.

`step-3-extend`



Act 3 — Extending the Sensor API



The Question



> "My sensor does something the Sensor API doesn't have.
> How do I add my own function?"

Usually, **you don't need to**. The API already has room built in:

Step	You want to expose	Mechanism
0	Standard readings	`sample_fetch` / `channel_get`
1	A standard setting	`attr_set` / `attr_get` (e.g. `SENSOR_ATTR_OFFSET`)
2	A value only your sensor has	Private channel: `SENSOR_CHAN_PRIV_START`
3	An action, or state that isn't a value	Device-specific functions in a public header

Zephyr®

A value → channel

A setting → attribute

An action → function



Step 1: Offset as a Standard Attribute




```c
static int aht10_attr_set(const struct device *dev, enum sensor_channel chan,
                          enum sensor_attribute attr, const struct sensor_value *val)
{
        struct aht10_data *data = dev->data;
        int64_t *offset = aht10_offset_for(data, chan);   /* temp or humidity */

        if (attr != SENSOR_ATTR_OFFSET || offset == NULL) {
                return -ENOTSUP;
        }

        k_mutex_lock(&data->lock, K_FOREVER);
        *offset = sensor_value_to_micro(val);
        k_mutex_unlock(&data->lock);
        return 0;
}
```

Added to `DEVICE_API(sensor, aht10_api)` as `.attr_set` and `.attr_get`. `channel_get` adds the offset.

Step 1b: A Default From Devicetree

```
# aosong,aht10.yaml
properties:
  temperature-offset-milli-celsius:
    type: int
    default: 0
```

```
env_sensor: aht10@38 {
          ...
          /* Board self-heating, measured once per hardware revision */
          temperature-offset-milli-celsius = <(-800)>;
};
```

```
.temp_offset_milli = DT_INST_PROP(inst, temperature_offset_milli_celsius),
```

DTS = per-board default. **Attribute** = per-unit adjustment at runtime.



Live: Fix the Unit On Site




```
uart:~$ sensor get aht10@38
channel type=13(ambient_temp) ... (23.500000)     ← reference says 22.0

uart:~$ sensor attr_get aht10@38 ambient_temp offset
aht10@38(channel=ambient_temp, attr=offset) value=-0.800000 ...   ← from devicetree

uart:~$ sensor attr_set aht10@38 ambient_temp offset -2.3       ← -0.8 + (22.0 - 23.5)
aht10@38 channel=ambient_temp, attr=offset set to value=-2.3

uart:~$ sensor get aht10@38
channel type=13(ambient_temp) ... (22.000000)
```

The command that failed in Act 1. **Same built-in shell, no shell code**: two more standard API functions.



Step 2: A Private Channel for Dew Point




```
/* include/aht10/aht10.h: public, shared by driver and app */
enum sensor_channel_aht10 {
          SENSOR_CHAN_AHT10_DEW_POINT = SENSOR_CHAN_PRIV_START,
};
```

```
/* in aht10_channel_get() */
} else if (chan == (enum sensor_channel)SENSOR_CHAN_AHT10_DEW_POINT) {
          ret = aht10_dew_point(data, val);   /* Magnus formula */
}
```

```
/* in the app: the same call as any other channel */
sensor_channel_get(dev, (enum sensor_channel)SENSOR_CHAN_AHT10_DEW_POINT, &dew);
```



Step 3: Device-Specific Functions




```c
/* include/aht10/aht10.h */
struct aht10_status {
          uint8_t raw;
          bool busy;
          bool calibrated;
};

bool aht10_is_device(const struct device *dev);
int  aht10_soft_reset(const struct device *dev);
int  aht10_get_status(const struct device *dev, struct aht10_status *status);
int  aht10_get_raw(const struct device *dev, uint32_t *hum, uint32_t *temp);
```

Reset, status and raw data aren't values or settings, so they get **functions**.
This is the pattern from the Zephyr docs: *"Device-Specific API Extensions"*.



The Guard: "Is This Really an AHT10?"




```
bool aht10_is_device(const struct device *dev)
{
        return dev != NULL && dev->api == &aht10_api;
}

int aht10_soft_reset(const struct device *dev)
{
        if (!aht10_is_device(dev)) {
                return -ENOTSUP;
        }
        ...
}
```

Any `const struct device *` can be passed in. Check it **before** touching `dev->data`.

Live: The Extensions From the App

```
/* app/src/main.c */
aht10_get_status(env_sensor, &status);                      /* device-specific API */

sensor_sample_fetch(env_sensor);
sensor_channel_get(env_sensor, SENSOR_CHAN_AMBIENT_TEMP, &temp);
sensor_channel_get(env_sensor, SENSOR_CHAN_HUMIDITY, &hum);
sensor_channel_get(env_sensor,
                   (enum sensor_channel)SENSOR_CHAN_AHT10_DEW_POINT, &dew);  /* private channel */
```

```
<inf> app: aht10@38 ready, status 0x18 (calibrated: yes)
<inf> app: temperature 21.20 C, humidity 63.10 %RH, dew point 13.90 C
```



The Anti-Pattern




```
/* Don't: "inherit" the sensor API and cast dev->api */
struct my_sensor_api {
          struct sensor_driver_api base;
          int (*reset)(const struct device *dev);
};

((const struct my_sensor_api *)dev->api)->reset(dev);
```

- `DEVICE_API(sensor, ...)` places the API in a typed linker section
- `DEVICE_API_IS(sensor, dev)` and every sensor tool rely on that
- It works for a single device, and then breaks the sensor shell, sensor lists, and any other device you cast by mistake

What the Built-in Shell Can Reach

Step	Built-in `sensor` shell?
0: `sample_fetch` / `channel_get`	✅ `sensor get`
1: `SENSOR_ATTR_OFFSET`	✅ `sensor attr_set` / `attr_get`
2: private dew point channel	❌ it doesn't know the name
3: `aht10_soft_reset()`, `aht10_get_status()`	❌ it doesn't know they exist

The further down the ladder, the more tooling **you** have to provide. That's Act 4.

`step-4-shell`



Act 4 — Our Own Shell



Command Tree




```
aht10
├── read      # temperature, humidity, dew point   (private channel)
├── status    # busy / calibrated bits              (aht10_get_status)
├── reset     # soft reset + re-init                (aht10_soft_reset)
└── raw       # 20-bit values, before conversion    (aht10_get_raw)
```

```
SHELL_STATIC_SUBCMD_SET_CREATE(sub_aht10,
          SHELL_CMD(read,   NULL, "Read temperature, humidity and dew point", cmd_read),
          SHELL_CMD(status, NULL, "Print the chip status byte", cmd_status),
          SHELL_CMD(reset,  NULL, "Soft-reset and re-initialize the chip", cmd_reset),
          SHELL_CMD(raw,    NULL, "Print the 20-bit raw values", cmd_raw),
          SHELL_SUBCMD_SET_END
);
SHELL_CMD_REGISTER(aht10, &sub_aht10, "AHT10 temperature and humidity sensor", NULL);
```

Offsets stay on the built-in `sensor attr_set`: they're **standard**.



A Handler Wraps One Extension




```c
static const struct device *const env_sensor = DEVICE_DT_GET(DT_NODELABEL(env_sensor));

static int cmd_status(const struct shell *sh, size_t argc, char **argv)
{
        struct aht10_status status;
        int ret;

        ret = aht10_get_status(env_sensor, &status);          /* device-specific API */
        if (ret < 0) {
                shell_error(sh, "status read failed (%d)", ret);
                return ret;
        }

        shell_print(sh, "status     0x%02x", status.raw);
        shell_print(sh, "calibrated %s", status.calibrated ? "yes" : "no");
        return 0;
}
```

Live: The Shell Reaches Everything

```
git checkout step-4-shell
west build -p -b frdm_mcxn236 app && west flash
```

```
uart:~$ aht10 read
temperature  21.20 C
humidity     63.10 %RH
dew point    13.90 C
uart:~$ aht10 status
status     0x18
busy       no
calibrated yes
uart:~$ aht10 reset
aht10@38 reset and re-initialized
```



Closing



What Goes Upstream



Back to option 2 from Act 1, now that Monday is safe:

What we built	Where it belongs
`attr_set` / `attr_get` with `SENSOR_ATTR_OFFSET`	**Upstream** in `dht20.c`: every user gains field calibration
Devicetree default offset	**Upstream**, same patch
Dew point private channel	Stays with us, or proposed as a generic helper
`aht10` shell	Stays in our app: it's product tooling

When upstream has the offset, we drop our driver and go back to `aosong,aht20`.

Takeaways

- **Check one layer at a time** with the built-in shell before writing any code
- When upstream falls short: **patch, upstream, or own driver**. Own it to ship, upstream it to stop owning it
- Implement the **standard API**, and every tool built on it works for free
- Extend in order: **attribute** → **private channel** → **device-specific function**
- A **value** is a channel, a **setting** is an attribute, an **action** is a function
- Extensions you add need **tools you provide**: a thin shell command per extension
- The **driver** protects its own data: it can't know which thread is calling

Thank You

Questions?





Resources



1. Device-Specific API Extensions
2. Sensor API Reference
3. Shell Documentation
4. Upstream driver: `drivers/sensor/aosong/dht20/dht20.c`
5. Full demo sources: `zephyr-fundamentals-lm3/` (one git tag per act)